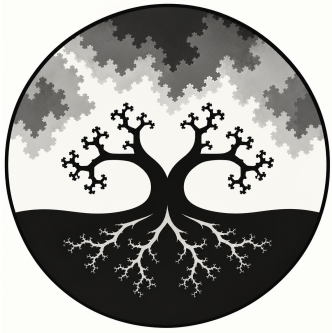


Angewandte Mathematik

Vorlesung 3 von 10

Mathematik in Lean

Skript: Abschnitte 3.2 und 3.6–3.8

Ablauf heute

0:00	Rückblick: Lean und Typentheorie	Vorlesung 2
0:07	Teil 1: Mengen in Lean	Skript 3.6
0:15	Taktiken am Beispiel der Mengen	Skript 3.6, 3.7
0:47	Zahlen und Rechnen	Skript 3.8
1:00	*Pause*	
1:15	Teil 2: Strukturen, Typklassen, Monoide	Skript 3.2
1:37	Die Strukturen der Mathematik in Lean	Skript 3.8
2:00	Zusammenfassung und Ausblick	
2:05	Lean live: Blatt 3, Teil B anfangen	Übungsblatt 3
2:30	Ende	

Roter Faden: Beweise mit Taktiken führen, am Beispiel der Mengen – und sehen, wie Mathlib aus Strukturen und Typklassen die Mathematik baut.

Rückblick: Vorlesung 2

- **Lean:** Typen abfragen, auswerten, definieren. Beweise als Term oder mit Taktiken – beide Wege liefern einen Term.
- Jeder Term hat einen Typ. Typen stehen in Universen: $\mathbb{N} : \mathrm{Type}$, $\mathrm{Type} : \mathrm{Type}\ 1$, … – daneben Prop, die Stufe der Aussagen.
- Abhängige Typen schreiben Voraussetzungen in den Typ, induktive Typen liefern Rekursion und Induktion.
- Gleichheit ist definitorisch (der Kern rechnet) oder propositional (mit Beweis). Dazu nur drei Axiome: äquivalente Aussagen sind gleich, Quotienten, Auswahl.
- Wenige Regeln: Var, Sort, Π-Form, λ, App, Conv und die der induktiven Typen. Der **Elaborator** ist groß und darf irren, der **Kern** ist klein und rechnet nur nach.
- **Curry–Howard:** Aussagen sind Typen, Beweise sind Terme; ohne Terme sind Einführung und Beseitigung Gentzens Regeln. Einen Beweis prüfen heißt, den Term typchecken.
- **Beweisirrelevanz:** Zwei Beweise derselben Aussage sind definitorisch gleich.

Vorlesung 3

Mengen in Lean

Mengen sind Prädikate

In Lean ist eine Menge von Elementen eines Typs α ein Prädikat auf α – definitorisch, nicht bloß ähnlich:

$$\mathrm{Set}\,\alpha \;\equiv\; (\alpha \to \mathrm{Prop}), \qquad (x \in s) \;\equiv\; s\,x, \qquad (x \in \{\, y \mid p\,y \,\}) \;\equiv\; p\,x.$$

- Alle drei sind definitorische Gleichheiten: Der Kern rechnet sie nach, bewiesen sind sie durch Reflexivität.
- $x \in s$ heißt: Das Prädikat s trifft auf x zu. Die Menge $\{\, y \mid p\,y \,\}$ ist das Prädikat p selbst.
- Eine Menge ist immer Teilmenge eines Typs, den es schon gibt. „Die Menge aller x mit $\varphi(x)$“ bildet man nur, wenn man sagt, woher die x kommen.

Mathlib. Mathlib definiert Mengen genau so, in `Mathlib/Data/Set/Defs.lean`. Mengenbeweise sind deshalb Logik, Element für Element.

Live in Lean `code/Vorlesung03.lean` – die drei Gleichungen, jede durch Reflexivität bewiesen

Mengenoperationen sind Logik

Hier und im Folgenden: α, β Typen, s, t, u Mengen in α, $f : \alpha \to \beta$. Für jedes $x : \alpha$ gilt

$$\begin{aligned} x \in s \cap t &\leftrightarrow x \in s \wedge x \in t \\ x \in s \cup t &\leftrightarrow x \in s \vee x \in t \\ x \in s^{\mathrm{c}} &\leftrightarrow \neg (x \in s) \\ x \in \emptyset &\leftrightarrow \mathrm{False} \\ s \subseteq t &\leftrightarrow \forall x,\ x \in s \to x \in t \\ y \in f(s) &\leftrightarrow \exists x,\ x \in s \wedge f\, x = y \end{aligned}$$

- Für ein festes x wird $\cap$ zu $\wedge$, $\cup$ zu $\vee$, das Komplement zu $\neg$ und $\emptyset$ zu False. $s \subseteq t$ wird $\forall$ mit $\to$, das Bild $f(s)$ wird $\exists$.
- Jede Zeile gilt definitorisch: Lean rechnet beide Seiten zum selben Term, der Beweis ist die Reflexivität. Deshalb sind die Taktiken für $\wedge$, $\vee$, $\neg$, $\to$, $\forall$, $\exists$ zugleich die Taktiken für Mengen.

Live in Lean `code/Vorlesung03.lean` – die sechs Zeilen, jede durch Reflexivität bewiesen

Vorlesung 3

Taktiken am Beispiel der Mengen

Teilmengen: Annahmen einführen und anwenden

Im Taktikmodus zeigt Lean einen *Beweiszustand*: die Annahmen, dann $\vdash$ und das Ziel. Eine *Taktik* ersetzt ihn durch einfachere – eine Schlussregel, rückwärts angewandt.

- **Einführen.** Ein Ziel $s \subseteq t$ heißt: Aus $x \in s$ folgt $x \in t$, für jedes x. Die Taktik intro nimmt x und die Annahme $h : x \in s$ in den Kontext:

$$\Gamma \vdash s \subseteq t \quad \rightsquigarrow \quad \Gamma,\ x : \alpha,\ h : x \in s \vdash x \in t.$$

- **Schließen.** Die Taktik exact schließt ein Ziel mit einem Beweis des passenden Typs. Ein Beweis von $x \in s \cap t$ ist ein Paar, seine erste Komponente beweist $x \in s$. So folgt $s \cap t \subseteq s$.
- **Anwenden.** Eine Annahme $h : s \subseteq t$ ist eine Funktion: Aus einem Beweis von $x \in s$ macht sie einen von $x \in t$, das x ergänzt Lean selbst. Die Taktik apply benutzt sie rückwärts: Statt $x \in t$ genügt $x \in s$.

Live in Lean `code/Vorlesung03.lean` – $s \cap t \subseteq s$ und die Transitivität von $\subseteq$, das Ziel nach jedem Schritt

Schnitt: zerlegen und zusammenbauen

Ein Beweis von $x \in s \cap t$ ist ein Paar: ein Beweis von $x \in s$ und einer von $x \in t$.

- **Annahme zerlegen.** Die Taktik obtain macht aus dem Paar seine beiden Teile:

$$\Gamma,\ h : x \in s \cap t \vdash Z \quad \rightsquigarrow \quad \Gamma,\ h_s : x \in s,\ h_t : x \in t \vdash Z.$$

- **Ziel zerlegen.** Die Taktik constructor macht aus dem Ziel zwei Ziele; Lean bearbeitet sie der Reihe nach:

$$\Gamma \vdash x \in t \cap s \quad \rightsquigarrow \quad \Gamma \vdash x \in t \quad \text{und} \quad \Gamma \vdash x \in s.$$

- **Kürzer.** Die Taktik rintro führt die Annahme ein und zerlegt sie in einem Schritt; das Paar (h_t, h_s) beweist das Ziel direkt. So folgt $s \cap t \subseteq t \cap s$.

Live in Lean `code/Vorlesung03.lean` – zwei Beweise von $s \cap t \subseteq t \cap s$, das Ziel nach jedem Schritt

Vereinigung: Fälle unterscheiden

Ein Beweis von $x \in s \cup t$ ist ein Beweis von $x \in s$ *oder* einer von $x \in t$, und man weiß, welcher.

- **Annahme benutzen.** Die Taktik rcases unterscheidet die beiden Fälle, jeden mit eigener Annahme und eigenem Ziel:

$$\Gamma,\ h : x \in s \cup t \vdash Z \quad \rightsquigarrow \quad \Gamma,\ h_s : x \in s \vdash Z \quad \text{und} \quad \Gamma,\ h_t : x \in t \vdash Z.$$

- **Ziel beweisen.** Bei einem Ziel $x \in t \cup s$ wählen left und right, welche Seite man beweist. Als Terme sind das die beiden Einbettungen in die Summe.
- Auch hier führt rintro ein und unterscheidet die Fälle in einem Schritt. So folgt $s \cup t \subseteq t \cup s$: im Fall $x \in s$ die rechte Seite, im Fall $x \in t$ die linke.

Live in Lean `code/Vorlesung03.lean` – zwei Beweise von $s \cup t \subseteq t \cup s$

Gleichheit von Mengen: Extensionalität

- Zwei Mengen sind gleich, wenn sie dieselben Elemente haben. Die Taktik ext führt eine Mengengleichung auf eine Äquivalenz für ein beliebiges x zurück:

$$\Gamma \vdash s \cap t = t \cap s \quad \rightsquigarrow \quad \Gamma,\ x : \alpha \vdash x \in s \cap t \leftrightarrow x \in t \cap s.$$

- Die Äquivalenz zerlegt constructor in zwei Richtungen. Jede ist eine Implikation und geht wie beim Schnitt: einführen, zerlegen, das Paar zusammenbauen.
- Oder man zeigt beide Inklusionen. Ein Satz der Bibliothek, die Antisymmetrie von $\subseteq$, setzt sie zu $s = t$ zusammen.
- Dahinter stehen zwei Prinzipien aus Vorlesung 2: Funktionen mit denselben Werten sind gleich, äquivalente Aussagen sind gleich.

Live in Lean `code/Vorlesung03.lean` – $s \cap t = t \cap s$ elementweise, dann Gleichheit aus zwei Inklusionen

Komplement und leere Menge

- $x \in s^{\mathrm{c}}$ ist $\neg(x \in s)$, also $x \in s \to \mathrm{False}$: eine Funktion. Mit $h : x \in s$ und $n : x \in s^{\mathrm{c}}$ ist die Anwendung $n\,h$ ein Beweis von False.
- $x \in \emptyset$ ist False. Deshalb schließt $n\,h$ auch das Ziel $x \in \emptyset$. So folgt $s \cap s^{\mathrm{c}} \subseteq \emptyset$.
- Ein Ziel $\neg P$ ist die Implikation $P \to \mathrm{False}$: intro nimmt P an, zu zeigen bleibt False,

$$\Gamma \vdash \neg P \quad \rightsquigarrow \quad \Gamma,\ h : P \vdash \mathrm{False}.$$

- $x \notin s$ steht für $\neg(x \in s)$. So folgt aus $x \in s$ die Aussage $x \notin s^{\mathrm{c}}$.

Live in Lean `code/Vorlesung03.lean` – $s \cap s^{\mathrm{c}} \subseteq \emptyset$ und $x \notin s^{\mathrm{c}}$

Bilder: Zeugen angeben und auspacken

$y \in f(s)$ heißt: Es gibt ein $x \in s$ mit $f\,x = y$. Ein Beweis davon besteht aus einem Zeugen x und den Beweisen von $x \in s$ und $f\,x = y$.

- **Zeugen angeben.** Beim Ziel $f\,x \in f(s)$ gibt die Taktik use den Zeugen x an. Was übrig bleibt, $x \in s$ und $f\,x = f\,x$, schließt use selbst: mit der Annahme und durch Reflexivität.
- **Auspacken.** Eine Annahme $y \in f(s)$ zerlegt rintro in den Zeugen x, den Beweis von $x \in s$ und die Gleichung $f\,x = y$. Die Gleichung setzt Lean sofort ein: y wird überall zu $f\,x$.
- So folgt aus $s \subseteq t$ die Inklusion $f(s) \subseteq f(t)$: Der Zeuge bleibt x, und $s \subseteq t$ macht aus dem Beweis von $x \in s$ einen von $x \in t$.

Live in Lean `code/Vorlesung03.lean` – $f\,x \in f(s)$ und $f(s) \subseteq f(t)$

Sätze finden

Mathlib benennt Sätze nach ihrer Aussage, Zeichen für Zeichen:

$\in$	$\cap$	$\cup$	$\subseteq$	s^c	$\emptyset$	$\leftrightarrow$	Voraussetzung
mem	inter	union	subset	compl	empty	iff	of

- inter_subset_left heißt „der Schnitt ist Teilmenge des linken Teils“: $s \cap t \subseteq s$.
- Vorn steht der Schluss, nach of die Voraussetzungen: mem_of_mem_of_subset macht aus $x \in s$ und $s \subseteq t$ die Aussage $x \in t$.
- Was ein Satz genau sagt, zeigt Lean auf Anfrage. Im Editor suchen zwei Taktiken, exact? und apply?, selbst nach einem passenden Satz.

Live in Lean `code/Vorlesung03.lean` – fünf Sätze über Mengen und was sie sagen

Umschreiben mit Gleichungen

- Die Taktik rw schreibt mit einer Gleichung um. Mit $h : s = t$ ersetzt sie im Ziel jedes s durch t und versucht danach, das Ziel durch Reflexivität zu schließen:

$$\Gamma \vdash s \cap u = t \cap u \quad \rightsquigarrow \quad \Gamma \vdash t \cap u = t \cap u \quad \rightsquigarrow \quad \text{bewiesen.}$$

- Die Gleichung darf ein Satz der Bibliothek sein, etwa $s \cap t = t \cap s$. Damit wird aus dem Ziel $s \cap t \subseteq t$ das Ziel $t \cap s \subseteq t$ – und das ist wieder ein Satz der Bibliothek.
- Umschreiben geht auch von rechts nach links, und auch in einer Annahme statt im Ziel.

Live in Lean `code/Vorlesung03.lean` – zwei Beweise durch Umschreiben

Vereinfachen mit Rechenregeln

- Die Taktik simp schreibt mit allen Gleichungen und Äquivalenzen um, die in der Bibliothek als Rechenregeln markiert sind, bis keine mehr greift.
- De Morgan, $(s \cup t)^{\mathrm{c}} = s^{\mathrm{c}} \cap t^{\mathrm{c}}$: Nach ext ist $x \in (s \cup t)^{\mathrm{c}} \leftrightarrow x \in s^{\mathrm{c}} \cap t^{\mathrm{c}}$ zu zeigen. simp macht daraus Aussagenlogik, $\neg(P \vee Q) \leftrightarrow \neg P \wedge \neg Q$, und erledigt sie.
- $s \cap \emptyset = \emptyset$ schafft simp allein.
- Auch eine Annahme vereinfacht simp. Wird sie zu False, ist alles bewiesen, etwa mit der Annahme $x \in s \cap \emptyset$. Mit einem Fragezeichen, simp?, zeigt Lean, welche Regeln simp benutzt hat.

Live in Lean `code/Vorlesung03.lean` – De Morgan, $s \cap \emptyset = \emptyset$ und ein Widerspruch aus $x \in s \cap \emptyset$

Ketten von Inklusionen

Mit calc schreibt man einen Beweis wie eine Rechnung auf Papier, jeden Schritt mit eigener Begründung:

$$
\begin{aligned}
s \cap u &\subseteq s && \text{der Schnitt ist Teilmenge des linken Teils} \\
&\subseteq t && \text{Voraussetzung } s \subseteq t \\
&\subseteq t \cup u && \text{jede Menge ist Teilmenge ihrer Vereinigung}
\end{aligned}
$$

- Lean setzt die Kette zu $s \cap u \subseteq t \cup u$ zusammen; die Transitivität von $\subseteq$ braucht man nicht zu nennen.
- Gleichungen und Inklusionen dürfen sich mischen, etwa $s \cap t = t \cap s \subseteq t$. Heraus kommt die Inklusion.
- Eine Kette mit calc geht als Beweisterm und als Taktik.

Live in Lean `code/Vorlesung03.lean` – die beiden Ketten

Ziel und Annahme: zwei Seiten einer Aussage

Im Beweiszustand stehen links von $\vdash$ die Annahmen, rechts das Ziel. Dieselbe Aussage kann an beiden Stellen stehen – daher die zwei Spalten der nächsten Tabelle.

- **Als Ziel** ist sie ein Auftrag: Man muss sie beweisen und arbeitet rückwärts – was genügt dafür?
- **Als Annahme** h ist sie ein Werkzeug: Man hat schon einen Beweis und arbeitet vorwärts – was folgt daraus?
- Ein Ziel ist erledigt, sobald es als Annahme dasteht (exact).

Taktik	**Beweiszustand danach**	**was passiert**
	$\vdash P \wedge Q \to Q \wedge P$	das Beispiel
intro	$h : P \wedge Q \vdash Q \wedge P$	Ziel $\to$: Annahme einführen
obtain	$p : P,\ q : Q \vdash Q \wedge P$	Annahme $\wedge$: zerlegen
constructor	$p : P,\ q : Q \vdash Q$ und $p : P,\ q : Q \vdash P$	Ziel $\wedge$: zwei Ziele
exact	–	beide Ziele stehen als Annahme da

Live in Lean `code/Vorlesung03.lean` – $P \wedge Q \to Q \wedge P$, das Ziel nach jedem Schritt

Eine Annahme $P \to Q$ benutzen: vorwärts und rückwärts

Eine Annahme $h : P \to Q$ ist eine Funktion: Sie macht aus einem Beweis von P einen Beweis von Q. Man kann sie auf zwei Arten benutzen.

- **Vorwärts:** Hat man schon $p : P$, so ist $h\,p : Q$ – h angewandt auf p.
- **Rückwärts:** Ist das Ziel Q, macht apply h daraus das Ziel P. Um Q zu zeigen, genügt P; den letzten Schritt erledigt h.

Taktik	**Beweiszustand danach**	**was passiert**
	$\vdash (P \to Q) \to (Q \to R) \to P \to R$	das Beispiel
intro	$f : P \to Q,\ g : Q \to R,\ p : P \vdash R$	dreimal annehmen
apply g	$f,\ g,\ p \vdash Q$	statt R genügt Q
apply f	$f,\ g,\ p \vdash P$	statt Q genügt P
exact p	–	P steht als Annahme da

Vorwärts ist derselbe Beweis ein Ausdruck: $g\,(f\,p) : R$. Bei Mengen ebenso: Mit $h : s \subseteq t$ macht apply h aus dem Ziel $x \in t$ das Ziel $x \in s$.

Live in Lean `code/Vorlesung03.lean` – die Kette von P nach R, rückwärts und vorwärts

Welche Taktik wann?

Form	**als Ziel beweisen**	**als Annahme h benutzen**
$P \to Q$	intro: P annehmen, Q zeigen	rückwärts: apply h, aus Ziel Q wird Ziel P vorwärts: aus $p : P$ wird $h\,p : Q$
$\forall x,\ P\,x$	intro: x einführen	einsetzen: $h\,a : P\,a$
$s \subseteq t$	intro: x und $x \in s$ annehmen	$h\,p : x \in t$
$P \wedge Q$, $x \in s \cap t$	constructor; oder das Paar (p, q)	obtain: zerlegen in $p : P$, $q : Q$
$P \vee Q$, $x \in s \cup t$	left oder right	rcases: zwei Fälle
$\neg P$, $x \in s^{\mathrm{c}}$	intro: P annehmen, False zeigen	$h\,p : \mathrm{False}$
$\exists x,\ P\,x$	use: Zeuge a, dann $P\,a$ zeigen	obtain: Zeuge x, Beweis von $P\,x$
$y \in f(s)$	use: Zeuge x	obtain: Zeuge x, y wird $f\,x$
$a = b$, $s = t$	Reflexivität, simp, calc; ext	rw, simp

- p ist ein Beweis von P, $h\,p$ die Anwendung von h auf p. Die mittlere Spalte ist die *Einführung*, die rechte die *Beseitigung* – die Regeln des natürlichen Schließens.
- Die Taktik exact schließt jedes Ziel, für das man schon einen Beweisterm hat.

Live in Lean `code/Vorlesung03.lean` – zu jeder Zeile ein Beispiel, als Ziel und als Annahme

Vorlesung 3

Zahlen und Rechnen

Am Anfang steht die Arithmetik

- Die Zahlbereiche sind gewöhnliche Typen. $\mathbb{N}$ ist der induktive Typ: Null und Nachfolger, sonst nichts. Darauf baut alles auf: $\mathbb{Z}$ aus n und $-(n+1)$ für $n \in \mathbb{N}$, $\mathbb{Q}$ als gekürzte Brüche, $\mathbb{R}$ über Cauchy-Folgen.
- **Konkrete Zahlen:** $17 \cdot 23 = 391$ ist eine Rechnung, keine Behauptung über alle Zahlen. Der Kern führt sie aus; der Beweis ist im Grunde das Ergebnis selbst.
  - $2 + 2 = 4$ durch Reflexivität: Der Kern rechnet beide Seiten aus.
  - $17 \cdot 23 = 391$ mit norm_num, $5 - 8 = -3$ in $\mathbb{Z}$ mit decide: Die Taktik baut den Beweisterm, der Kern prüft ihn durch Rechnen.
  - Auswerten ohne Beweis: Lean rechnet 2^{10} aus und zeigt 1024.

Live in Lean `code/Vorlesung03.lean` – die vier Zahlbereiche und die vier Rechnungen

Drei Taktiken für die Praxis

Taktik	wofür	Beispiel
ring	Gleichungen in kommutativen (Halb-)Ringen, etwa $\mathbb{N}$	$(a+b)^2 = a^2 + 2ab + b^2$
omega	lineare Arithmetik über $\mathbb{N}$, $\mathbb{Z}$, mit Ungleichungen	aus $3 < n$ folgt $2 < n$
norm_num	konkrete Zahlen, auch in $\mathbb{Q}$ und $\mathbb{R}$	$\frac{1}{3} + \frac{1}{6} = \frac{1}{2}$

- Die binomische Formel gilt für *alle* a, b; ring beweist sie als Gleichung im kommutativen Ring $\mathbb{Z}$.
- In der zweiten Zeile ist $3 < n$ die Voraussetzung; das Ziel $2 < n$ folgt daraus linear.
- Probe zur letzten Zeile: $\frac{1}{3} + \frac{1}{6} = \frac{2}{6} + \frac{1}{6} = \frac{3}{6} = \frac{1}{2}$.

Live in Lean `code/Vorlesung03.lean` – die drei Beispiele der Tabelle

Aussagen über alle Zahlen: Induktion

- $n + 0 = n$ gilt durch Reflexivität, $0 + n = n$ nicht. Denn die Addition auf $\mathbb{N}$ rekursiert über das *zweite* Argument: $n + 0 \equiv n$ und $n + (m + 1) \equiv (n + m) + 1$. Bei $n + 0$ greift die erste Gleichung, bei $0 + n$ steht dort die Variable n – keine greift.
- Bei $0 + n$ bleibt die Rechnung also an der Variablen n hängen. Für jedes n beweist man die Gleichung deshalb durch Induktion über n:

$$0 + 0 \equiv 0, \qquad 0 + (k + 1) \equiv (0 + k) + 1 = k + 1 \quad \text{nach Induktionsannahme.}$$

- In Lean steht der Beweis genau so da: induction liefert Anfang und Schritt, der Schritt ist die Kette oben – erst die Definition, dann die Induktionsannahme.

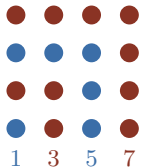
Live in Lean `code/Vorlesung03.lean` – $n + 0 = n$ durch Rechnen, $0 + n = n$ durch Induktion

Ein Induktionsbeweis: die Summe der ungeraden Zahlen

Aussage: $1 + 3 + 5 + \cdots + (2n - 1) = n^2$ für alle $n \in \mathbb{N}$.
Die Summe $S(n)$ definieren wir selbst, rekursiv: $S(0) \equiv 0$ und $S(k + 1) \equiv S(k) + (2k + 1)$.

- **Anfang:** $S(0) = 0 = 0^2$ – das rechnet Lean aus.
- **Schritt:** Mit der Induktionsannahme $S(k) = k^2$ gilt

$$\begin{aligned} S(k+1) &\equiv S(k) + (2k+1) && \text{(1) Definition} \\ &= k^2 + 2k + 1 && \text{(2) Induktionsannahme} \\ &= (k+1)^2 && \text{(3) binomische Formel} \end{aligned}$$



$1 + 3 + 5 + 7 = 4^2$: Jeder Winkel macht das Quadrat breiter.

- Dasselbe Muster wie bei $0 + n = n$: erst die Definition, dann die Induktionsannahme, den Rest rechnet ring. Mit Mathlibs Summenzeichen genauso – so auf Blatt 3.

Live in Lean `code/Vorlesung03.lean` – die Summe und die Formel, auch mit Mathlibs Summe

Pause

Danach: Strukturen und Typklassen, die Strukturen der Mathematik – dann Lean live mit Blatt 3

Vorlesung 3

Strukturen und Typklassen

Strukturen: Daten bündeln

- Eine *Struktur* legt einen neuen Typ an: Tupel mit benannten *Feldern*. Ein Punkt besteht aus zwei ganzen Zahlen, den Feldern x und y – wie $\mathbb{Z} \times \mathbb{Z}$, nur mit Namen.
- Gebaut wird ein Punkt aus seinen Feldern, etwa $A = (3, 4)$ und $B = (1, -2)$. Gelesen wird über die Feldnamen: das x von A plus das y von B ergibt $3 + (-2) = 1$.
- Felder dürfen Beweise sein: Eine positive Zahl ist eine Zahl $n \in \mathbb{N}$ zusammen mit einem Beweis von $0 < n$. Ohne den Beweis lässt sich keine bauen; für die 3 liefert ihn norm_num.
- So wird $\{\, n \in \mathbb{N} \mid 0 < n \,\}$ ein Typ: Jedes Element trägt seinen Beweis mit.

Live in Lean `code/Vorlesung03.lean` – die Punkte A und B, die positive Zahl 3

Strukturen und Typklassen

Begriff	was es ist
induktiver Typ	Konstruktoren angeben; Lean baut den Rekursor
Struktur	ein induktiver Typ mit *einem* Konstruktor
Typklasse	eine Struktur, deren Instanzen Lean *selbst sucht*

- Darauf beruht der Aufbau von Mathlib: Die ganze algebraische Hierarchie, von der Halbgruppe über den Ring bis zum Körper, besteht aus Typklassen, die auseinander hervorgehen.

Mathlib. Wer in Mathlib einen Satz über alle Körper beweist, nimmt einen Typ K und dazu die Körperstruktur als *Instanzargument* an, in eckigen Klammern. Ein Instanzargument sucht Lean selbst. Deshalb reicht es, $\mathbb{R}$ zu nennen – die Körperstruktur kommt von allein dazu.

Eine Typklasse und eine Instanz

- Ein *Monoid* ist ein Typ M mit einer assoziativen Verknüpfung $\cdot$ und einem neutralen Element e, etwa $\mathbb{N}$ mit $+$ und 0.
- Als Typklasse hat es fünf Felder: zwei *Daten*, die Verknüpfung und e, und drei *Gesetze* – Felder, deren Typ eine Aussage ist:

$$(a \cdot b) \cdot c = a \cdot (b \cdot c), \qquad e \cdot a = a, \qquad a \cdot e = a \qquad \text{für alle } a, b, c \in M.$$

- Eine *Instanz* füllt die Felder für einen bestimmten Typ. Für $\mathbb{N}$: die Addition, die 0 und als Gesetze drei Sätze der Bibliothek, die Assoziativität der Addition, $0 + a = a$ und $a + 0 = a$.

Live in Lean `code/Vorlesung03.lean` – die Typklasse mit ihren fünf Feldern, die Instanz für $\mathbb{N}$

Instanzsuche

- Ein Satz für *jedes* Monoid: $(a \cdot e) \cdot e = a$. Der Beweis schreibt zweimal mit dem Gesetz $a \cdot e = a$ um: $(a \cdot e) \cdot e \ \to \ a \cdot e \ \to \ a$.
- Die Monoidstruktur ist ein *Instanzargument*: Wer den Satz für $\mathbb{N}$ benutzt, übergibt sie nicht – Lean findet die Instanz selbst. Für $\mathbb{N}$ sagt der Satz $(a + 0) + 0 = a$.
- Ebenso sucht und findet Lean die echten Strukturen von Mathlib: $\mathbb{Z}$ als additive Gruppe und als kommutativen Ring, $\mathbb{R}$ als Körper.

Live in Lean `code/Vorlesung03.lean` – der Satz für jedes Monoid, dann die Suche nach den Strukturen von $\mathbb{Z}$ und $\mathbb{R}$

Ein Satz, viele Monoide

Dieselbe Typklasse, drei Instanzen:

Typ	**Verknüpfung**	**neutral**	**Gesetze**
$\mathbb{N}$	$+$	0	Sätze der Bibliothek
Listen	Aneinanderhängen	leere Liste	Sätze der Bibliothek
Funktionen $\alpha \to \alpha$	$\circ$	id	definitorisch, durch Reflexivität

- Bei den Funktionen rechnen $(f \circ g) \circ h$ und $f \circ (g \circ h)$ zum selben Term; ebenso $\mathrm{id} \circ f$ und $f \circ \mathrm{id}$ zu f.
- Der Satz $(a \cdot e) \cdot e = a$ gilt ohne neuen Beweis für jede Instanz. Für Listen heißt er: Hängt man an l zweimal die leere Liste an, kommt l heraus.

Live in Lean `code/Vorlesung03.lean` – die Instanzen für Listen und Funktionen, der Satz für Listen

Ein Beispiel aus der Informatik: Zusammenfassen in Teilen

Zusammenfassen (in Programmiersprachen fold oder reduce) verknüpft alle Elemente einer Liste: $\mathrm{zus}(x_1, \ldots, x_n) = x_1 \cdot x_2 \cdots x_n$; die leere Liste ergibt e. Das ist *eine* Definition für jedes Monoid.

Satz (Teilen und Zusammenfassen)

In jedem Monoid gilt $\mathrm{zus}(x_1, \ldots, x_k, y_1, \ldots, y_m) = \mathrm{zus}(x_1, \ldots, x_k) \cdot \mathrm{zus}(y_1, \ldots, y_m)$.

- Beweis: Induktion über die erste Liste. Gebraucht werden nur das neutrale Element und die Assoziativität.
- Darauf beruht MapReduce: Die Daten werden auf viele Rechner verteilt, jeder fasst seinen Teil zusammen, am Ende werden die Teilergebnisse verknüpft – mit garantiert demselben Ergebnis wie auf einem Rechner.
- Zählen ($\mathbb{N}$ mit $+$), Listen anhängen, Texte aneinanderfügen: Für Texte genügt eine neue Instanz, dann gilt der Satz sofort.

Live in Lean `code/Vorlesung03.lean` – Zusammenfassen und Teilen, die Instanz für Zeichenketten

Erweitern: vom Monoid zur Gruppe

- Eine Gruppe ist ein Monoid mit Inversen. Die Typklasse der Gruppen *erweitert* die der Monoide (in Lean: extends). Sie übernimmt alle fünf Felder und fügt zwei hinzu: die Abbildung $a \mapsto a^{-1}$ und das Gesetz $a^{-1} \cdot a = e$.
- Die Instanz für $\mathbb{Z}$ gibt alle Felder an, auch die übernommenen: $+$, 0, die drei Gesetze, dazu $a \mapsto -a$ mit dem Gesetz $-a + a = 0$; das beweist omega.
- Jede Gruppe ist damit von selbst ein Monoid: Zur Gruppe $\mathbb{Z}$ findet die Instanzsuche auch das Monoid, und $(a \cdot e) \cdot e = a$ gilt ohne neuen Beweis, als $(a + 0) + 0 = a$.

Live in Lean `code/Vorlesung03.lean` – die Typklasse der Gruppen, die Instanz für $\mathbb{Z}$, der Satz über Monoide für $\mathbb{Z}$

Ein einfaches Beispiel aus der Mathematik: Bits mit XOR

$\oplus$	0	1
0	0	1
1	1	0

Die Bits $\{0, 1\}$ mit XOR, also Addition modulo 2, bilden eine Gruppe: $\oplus$ ist assoziativ, 0 ist neutral, und jedes Bit ist sein eigenes Inverses.

- In jeder Gruppe macht b^{-1} die Verknüpfung mit b rückgängig: $b^{-1} \cdot (b \cdot a) = a$. Beweis: Assoziativität, dann $b^{-1} \cdot b = e$, dann $e \cdot a = a$.
- Für Bits heißt das $k \oplus (k \oplus x) = x$: Verschlüsseln mit dem Schlüssel k und Entschlüsseln mit demselben Schlüssel (One-Time-Pad).
- In Lean genügt eine Instanz für die Wahrheitswerte. Das Entschlüsseln folgt dann aus dem Satz für alle Gruppen, ohne neuen Beweis.

Live in Lean `code/Vorlesung03.lean` – die Instanz für Bits und der Satz über das Rückgängigmachen

Vorlesung 3

Die Strukturen der Mathematik in Lean

Die algebraische Hierarchie

Die Grundstrukturen bauen aufeinander auf, jede erweitert die vorige:

$$\text{Halbgruppe} \to \text{Monoid} \to \text{Gruppe}$$

$$\text{Monoid},\ \text{abelsche Gruppe} \to \text{Ring} \to \text{kommutativer Ring} \to \text{Körper}$$

- Ein Ring ist für $+$ eine abelsche Gruppe, für $\cdot$ nur ein Monoid. Sein Weg führt deshalb nicht über die Gruppe.
- Jede Stufe übernimmt die Felder der vorigen: Ein Körper ist auch ein kommutativer Ring und ein Ring. Deshalb findet die Instanzsuche zu einem Körper auch den Ring – jeder Satz über Ringe gilt für jeden Körper.
- **Ein Satz für alle Gruppen:** $a \cdot a^{-1} = 1$, mit der Gruppenstruktur als Instanzargument. Man beweist ihn einmal; Lean setzt ihn für jede Gruppe ein, etwa für die Permutationen von drei Elementen.

Live in Lean `code/Vorlesung03.lean` – die Stufen der Hierarchie, der Satz für alle Gruppen

Halbring, Ring, Körper: was die Stufen verlangen

Eine Verknüpfung: Halbgruppe (assoziativ), Monoid (dazu ein neutrales Element), Gruppe (dazu Inverse); *abelsch* heißt vertauschbar. Bei zwei Verknüpfungen $+$ und $\cdot$ kommen $a(b+c) = ab + ac$ und $(a+b)c = ac + bc$ dazu, außerdem $0a = a0 = 0$:

Stufe	**verlangt**	**Beispiele**
Halbring	$(+, 0)$ abelsches Monoid, $(\cdot, 1)$ Monoid	$\mathbb{N}$
Ring	dazu Negative: $(+, 0)$ ist eine abelsche Gruppe	Matrizen, $AB \neq BA$
kommutativer Ring	dazu $ab = ba$	$\mathbb{Z}$, 64-Bit-Zahlen
Körper	dazu Inverse: $aa^{-1} = 1$ für $a \neq 0$; $0 \neq 1$	$\mathbb{Q}$, $\mathbb{R}$, Bits

- $\mathbb{N}$ hat keine Negativen; Lean schneidet deshalb ab: $3 - 5 = 0$.
- 64-Bit-Zahlen rechnen mit Überlauf, also modulo 2^{64}. Kein Körper: $2 \cdot 2^{63} = 0$, also hat 2 kein Inverses.
- Bits mit XOR als $+$ und AND als $\cdot$ bilden den kleinsten Körper, $\mathbb{F}_2$. Mit ihm rechnen Prüfsummen und fehlerkorrigierende Codes.

Live in Lean `code/Vorlesung03.lean` – zu jeder Stufe ein Beispiel und die Rechnungen

Vektorräume, wie man sie kennt

Wie bei den Monoiden: erst die Begriffe aus der Linearen Algebra selbst bauen, dann der Abgleich mit Mathlib.

Definition (Vektorraum über einem Körper K)

$(V, +)$ ist eine abelsche Gruppe, dazu kommt eine Skalarmultiplikation $(a, v) \mapsto av$ mit

$$a(u+v) = au + av, \qquad (a+b)v = av + bv, \qquad (ab)v = a(bv), \qquad 1v = v.$$

- In Lean eine Typklasse mit genau diesen vier Gesetzen; $+$ und die Skalarmultiplikation liefern die Klassen davor.
- Standardbeispiel K^n: Ein Vektor ist eine Funktion, die jedem Index i die Komponente v_i zuordnet; gerechnet wird komponentenweise.
- Die erste Rechnung, von Hand: $0v + 0v = (0+0)v = 0v$, also $0v = 0$. Ebenso $a0 = 0$.

Live in Lean `code/Vorlesung03.lean` – die Klasse Vektorraum, die Instanz für K^n, $0v = 0$

Definition (linear)

$f\colon V \to W$ heißt linear, wenn $f(u+v) = f(u) + f(v)$ und $f(av) = af(v)$ für alle $u, v \in V$ und $a \in K$.

- Dann ist $f(0) = f(0 \cdot 0) = 0 \cdot f(0) = 0$ – mit der ersten Rechnung.
- Matrix mal Vektor, $(Av)_i = \sum_j a_{ij} v_j$, ist linear. Beispiel: Die Drehung um $90°$ bildet $(1, 0)$ auf $(0, 1)$ ab; Lean rechnet es nach.
- Der Kern $\{v \mid f(v) = 0\}$ ist eine Menge, also ein Prädikat wie im ersten Teil. Er ist ein Untervektorraum: Er enthält 0 und ist abgeschlossen unter $+$ und der Skalarmultiplikation.

Live in Lean `code/Vorlesung03.lean` – linear, Matrix mal Vektor, die Drehung, der Kern

Basis, Dimension und Mathlib

- $v_1, \ldots, v_n$ sind linear unabhängig, wenn aus $\sum_i c_i v_i = 0$ folgt $c_1 = \cdots = c_n = 0$. Die Einheitsvektoren $e_1, \ldots, e_n$ sind es: Die j-te Komponente von $\sum_i c_i e_i$ ist c_j. Sie bilden die Standardbasis, und $\dim K^n = n$.
- In Mathlib heißt ein Vektorraum *Modul* über einem Körper. Eine eigene Klasse gibt es nicht; so gilt jeder Satz über Moduln auch für Vektorräume. „Sei V ein K-Vektorraum“ sind dort drei Instanzargumente: Körper, abelsche Gruppe, Modul.
- Lean prüft den Abgleich: Jedes Modul über einem Körper ist ein Vektorraum in unserem Sinn und umgekehrt; Mathlib verlangt dazu noch $0v = 0$ und $a0 = 0$, die erste Rechnung. Ebenso stimmen linear, Kern, linear unabhängig und die Standardbasis überein.

Live in Lean `code/Vorlesung03.lean` – die Namen in Mathlib und der Abgleich

Ordnungen

Ordnungen bilden eine Hierarchie von Typklassen:

$$\text{Präordnung} \to \text{Halbordnung} \to \text{lineare Ordnung}$$
$$\text{Verband} \to \text{vollständiger Verband}$$

- Die Teilmengen eines Typs bilden einen *vollständigen Verband*: geordnet durch $\subseteq$, und jede Familie von Mengen hat ein Infimum, ihren Schnitt, und ein Supremum, ihre Vereinigung.
- Die Instanzsuche findet diese Struktur für die Mengen natürlicher Zahlen, also für die Prädikate auf $\mathbb{N}$.

Live in Lean `code/Vorlesung03.lean` – die Stufen der Ordnungen, der Verband der Teilmengen von $\mathbb{N}$

$\mathbb{R}$ rechnet nicht

- $\mathbb{N}$ ist berechenbar: Lean rechnet 2^{10} aus und zeigt 1024.
- Bei $\mathbb{R}$ bekommt man so keine Zahl. Eine reelle Zahl ist dort durch eine unendliche Folge von Näherungen gegeben, und ob zwei solche Folgen dieselbe Zahl beschreiben, kann kein Programm entscheiden.
- Deshalb sind Vergleiche und Division auf $\mathbb{R}$ als nicht berechenbar markiert. Addition und Multiplikation lassen sich zwar auswerten, liefern aber nur eine Folge von Näherungen, keine Zahl.
- Statt decide nimmt man norm_num. Für das Beweisen ändert das nichts – nur ausrechnen lassen kann man sich nichts.

Bemerkung. Die Instanzsuche findet die lineare Ordnung auf $\mathbb{R}$. Sie ist nicht berechenbar – für Beweise steht sie trotzdem bereit.

Live in Lean `code/Vorlesung03.lean` – am Ende der Zahlen: die lineare Ordnung auf $\mathbb{R}$

Mixins: kombinieren statt vererben

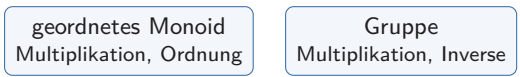
Ein geordnetes kommutatives Monoid ist ein kommutatives Monoid *und* eine Ordnung *und* die Verträglichkeit von beidem: Aus $a \leq b$ folgt $c \cdot a \leq c \cdot b$.

- Man könnte eine Klasse bauen, die von beiden erbt. Mathlib macht es nicht so, sondern nimmt drei getrennte Instanzargumente: kommutatives Monoid, Halbordnung, Verträglichkeit.
- Die dritte Klasse trägt selbst keine Daten – sie ist eine Aussage und sagt nur, dass Multiplikation und Ordnung zusammenpassen. So eine datenfreie Zusatzklasse heißt *Mixin*.
- Der Beweis in Lean: Die Taktik gcongr führt $c \cdot a \leq c \cdot b$ auf $a \leq b$ zurück – und das ist die Voraussetzung.

Live in Lean `code/Vorlesung03.lean` – die drei Instanzargumente und der Beweis von $c \cdot a \leq c \cdot b$

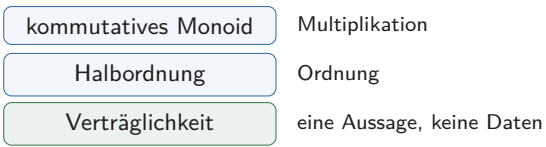
Warum Mixins?

gebündelt: zwei Klassen zusammen



zwei Multiplikationen auf M,
Lean weiß nicht, dass sie gleich sind

Mixin – so macht es Mathlib



- Gebündelt trägt jede Klasse ihre eigene Multiplikation. Setzt man eine geordnete Gruppe aus einem geordneten Monoid und einer Gruppe zusammen, hat M zwei Multiplikationen, und Lean weiß nicht, dass sie gleich sind. Sonst bräuchte man für jede Kombination eine eigene Klasse.
- Mit Mixins kommt die Multiplikation genau einmal, aus dem kommutativen Monoid; die Verträglichkeit fügt nur eine Eigenschaft hinzu.

Mathlib. Daran erkennt man Mathlib-Code: eine Reihe von Instanzargumenten, für einen Vektorraum etwa ein Körper K, eine abelsche Gruppe V und die Modulstruktur. Das sagt genau, welche Struktur der Beweis braucht – und nicht mehr.

Unterobjekte und Morphismen sind gebündelt

Unterobjekte. Eine Untergruppe von G ist ein Element eines eigenen Typs, nicht bloß eine Teilmenge mit einer Eigenschaft daneben.

eine Untergruppe U von G
die Trägermenge
dazu Beweise:
– enthält das neutrale Element
– abgeschlossen unter Produkt
– abgeschlossen unter Inversem

Deshalb kann man mit Untergruppen rechnen, etwa Schnitte und Erzeugnisse bilden.

Morphismen. Ebenso ein Homomorphismus $f : G \to H$: keine Funktion mit einer Bedingung daneben, sondern ein Element eines eigenen Typs.

ein Homomorphismus f von G nach H
die Funktion
dazu Beweise:
– $f(1) = 1$
– $f(a \cdot b) = f(a) \cdot f(b)$

Angewendet wird er wie eine Funktion; die Umwandlung fügt Lean selbst ein (Koerzierung).

Live in Lean `code/Vorlesung03.lean` – Untergruppen und Untermoduln; Homomorphismen, lineare und stetige lineare Abbildungen; die Homomorphie-Gleichung

Zusammenfassung

- Mengen sind Prädikate: $\mathrm{Set}\,\alpha$ ist $\alpha \to \mathrm{Prop}$. Für festes x wird $\cap$ zu $\wedge$, $\cup$ zu $\vee$, das Komplement zu $\neg$; $\subseteq$ wird $\forall$ mit $\to$, das Bild $\exists$.
- Deshalb sind Mengenbeweise Logik. Eine Taktik ersetzt den Beweiszustand Annahmen $\vdash$ Ziel durch einfachere: Annahmen einführen, anwenden, zerlegen, in Fälle teilen; Ziele zusammenbauen, eine Seite wählen, einen Zeugen angeben; Mengen elementweise vergleichen.
- Umschreiben, vereinfachen, in Ketten rechnen. Für Zahlen: ring, omega, norm_num und Induktion.
- Strukturen bündeln Daten und Beweise. Typklassen sind Strukturen, deren Instanzen Lean selbst sucht: Ein Satz über Monoide gilt für $\mathbb{N}$, Listen und Funktionen; Erweitern baut Stufen aufeinander auf.
- Daraus baut Mathlib die Mathematik: algebraische Hierarchie, lineare Algebra, Ordnungen, Mixins. Unterobjekte und Morphismen sind gebündelte Typen.

Nächstes Mal: Räume und Konvergenz

- Abschnitte 4.1–4.2: die Räume der Analysis – nach demselben Muster gebaut wie die Algebra.
- Vier Stufen: topologischer, metrischer, normierter Raum, Innenproduktraum. Jede liefert die darunterliegenden mit, und die Instanzsuche findet die Instanzen selbst.
- Konvergenz durch *Filter*: ein Begriff statt vieler Grenzwertdefinitionen – atTop, $\mathrm{Tendsto}$ und die ε-N-Form.

Lean live: Blatt 3, Teil B

- `lean/Blatt03.lean` in VS Code öffnen und jede Lücke, jedes sorry, durch einen Beweis ersetzen.
- Die Taktiken stehen in der Tabelle „Welche Taktik wann?“ aus dem ersten Teil; dazu Induktion, ring und decide für die Zahlen.
- Das Infofenster zeigt nach jedem Schritt den Beweiszustand: Annahmen und Ziel.
- Anfangen mit B1: Mengen sind Prädikate.

Live in Lean `code/Vorlesung03.lean` – zum Nachschlagen: jede Taktik an einem Beispiel, in der Reihenfolge der Folien